

Desafio Técnico — Backend Python + QA

Vaga: Backend Junior/Pleno com skills em QA · Tempo estimado: **3 a 4 horas** · Prazo de entrega: **5 dias corridos**

Contexto

A **Have I Been Pwned (HIBP)** mantém um catálogo público de *data breaches*: vazamentos de dados confirmados em serviços da internet, com os metadados de cada incidente (domínio afetado, data do vazamento, volume de contas comprometidas e as classes de dados expostas — e-mails, senhas, cartões etc.). Sua missão é construir o **Breach Radar**: uma API que consome o catálogo da HIBP e expõe consultas com filtros.

 **Feed oficial:** <https://haveibeenpwned.com/api/v3/breaches>

O endpoint `/breaches` é **público (sem API key)** e retorna um **array JSON** com todos os breaches do catálogo (~800+). A HIBP **exige** um header `User-Agent` na requisição — sem ele a resposta é `403`. Cada item traz, entre outros campos: `Name` (identificador único do breach, ex.: `Adobe`, `LinkedIn`), `Domain`, `BreachDate` (`YYYY-MM-DD`), `AddedDate` (ISO 8601 com hora), `PwnCount` (inteiro), `DataClasses` (lista de strings) e flags booleanas (`IsVerified`, `IsSensitive`, `IsSpamList` ...).

Parte 1 — API

REST API em **Python + PostgreSQL** com **Swagger/OpenAPI** exposto em `/docs`.

Endpoints

Método	Rota	Descrição
POST	<code>/sync</code>	Sincroniza com o feed da HIBP
GET	<code>/breaches</code>	Lista breaches com filtros (ver abaixo)
GET	<code>/breaches/{name}</code>	Detalhe de um breach

Filtros em `GET /breaches` 🌟

A **qualidade dos filtros é o principal critério** desta parte:

- `domain` — case-insensitive, match parcial
- `name` — busca exata (chave do breach); valida formato de slug → 400 se inválido
- `breach_date_from` / `breach_date_to` — janela da **data do vazamento** (`BreachDate`), inclusiva
- `added_date_from` / `added_date_to` — janela de **quando entrou no catálogo** (`AddedDate`)
- `data_class` — match **case-insensitive** em `DataClasses` (ex.: `Passwords` , `Email addresses`)
- `min_pwn_count` / `max_pwn_count` — faixa sobre `PwnCount` (inteiros ≥ 0)
- `is_verified` / `is_sensitive` / `is_spam_list` — flags booleanas (`true` / `false`)

Paginação: estratégia à sua escolha (`page` / `page_size` , `limit` / `offset` , `cursor` etc.) — justifique a decisão no README.

Combinações de filtros devem funcionar juntas (semântica E / AND).

Regras

1. **Idempotência:** rodar `/sync` várias vezes não duplica breaches (use `Name` como chave).
2. **Resiliência:** se o feed cair, a API continua respondendo do cache local.
3. **Validação:** parâmetros malformados (data fora do formato `YYYY-MM-DD` , `pwn_count` não-inteiro ou negativo, `name` com caracteres inválidos) → 400 com mensagem clara.

🔧 Parte 2 — QA

2.1 Testes (cobertura mínima 80%)

Mocke o feed externo (`respx` , `responses` ou similar — **sem chamadas reais à HIBP em CI**). Cenários esperados:

- Feed fora do ar (timeout / 500)
- JSON malformado ou campos faltando (ex.: breach sem `DataClasses` , sem `Domain` ou

`sem BreachDate )`

- `/sync` rodado 2x → sem duplicação
- Cada filtro isolado e combinações
- Parâmetro inválido → 400
- Paginação nas bordas (primeira página, última, valor exagerado)

2.2 Plano de testes (`TEST_PLAN.md`)

- Estratégia adotada
- Casos de teste positivos, negativos e edge cases
- O que **não** foi testado e por quê
- Riscos identificados na aplicação

2.3 Bug hunt 🐛

Em `legacy/breach_matcher.py` há **3 bugs plantados**. Para cada um:

1. Documente em `legacy/BUGS_FOUND.md` : o que está errado, como reproduzir, severidade e impacto em produção (somos cybersec — **um vazamento crítico sumindo do índice é um incidente**).
2. Escreva um teste que **falhe** expondo o bug.
3. Corrija e garanta que o teste passa.

★ Parte 3 — Opcionais (contam pontos extras)

Nenhum item é necessário para passar, mas serão levados em conta na avaliação:

- 🐳 Docker + docker-compose (app + Postgres subindo com 1 comando)
- 🤖 CI configurado (GitHub Actions com lint + testes + cobertura)
- 🧬 Migrations versionadas (Alembic ou similar)
- 🕒 Agendamento automático da sync (cron / APScheduler)
- 📄 Logs estruturados em JSON
- ♻️ Cache HTTP com `ETag` / `If-None-Match`

Entrega

- Repositório Git público (ou .zip)
- `README.md` com setup, como rodar testes e decisões técnicas
- Enviar para [seu-email@empresa.com]

Avaliação

Critério	Peso
Endpoints e filtros funcionando	25%
Qualidade dos testes	30%
Bug hunt (identificação + impacto)	25%
Código limpo + documentação	20%
Opcionais entregues	até +10% extra

Dúvidas? Manda no email. Suposições valem — documente no README. **Boa sorte!** 